Lucas Vieira
INFO3-FA

IUT de Vélizy-Rambouillet
CAMPUS DE VÉLIZY-VILLACOUBLAY

Rapport – Qualité de développement :

Année 2025/2026

Lucas Vieira
INFO3-FA

I - Introduction :

Notre objectif est d'analyser le code Monte Carlo Master-Worker, ce code est composé de 2 fichiers dont un fichier qui concerne le Master puis un autre fichier qui concerne le Worker.

Master-Worker est un paradigme de programmation parallèle, son fonctionnement est assez simple, il y a un maitre qui s'occupe de répartir des tâches à des travailleurs dans l'objectif que cela aille plus vite notamment grâce au parallélisme.

Ici, on souhaite évaluer les critères de qualité de ce code plus précisément l'efficience à travers du temps, pour cela nous allons donc nous intéresser au speed up à travers de 2 courbes que nous détaillerons plus tard.

A – L'environnement de test :

Afin de pouvoir effectuer nos tests, il est nécessaire d'avoir un ordinateur à porter de main, dans notre cas nous avons la machine suivante :

- 1 ordinateur Dell Optiplex 7050
    - Intel Core i7-7700 3,6Ghz composée de 4 cœurs physiques et 8 processeurs logiques
    - 32 Go de RAM

B – Adaptation du code :

Comme indiqué plus haut, on s'intéresse donc au temps d'exécution de notre code, de ce fait nous allons donc dans un premier temps modifier l'unité de temps afin de gagner en précision car pour le moment nous utilisons des millisecondes, nous allons donc remplacer cela par des nanosecondes.

Maintenant que cela est fait, il faut que l'on s'intéresse à l'automatisation du code, dans l'énoncée il est indiqué qu'il faudra dans nos tests entre 1 à 12 workers. Nous allons donc effectuer nos tests en utilisant 4 workers.

II – Scalabilité forte :

La scalabilité forte mesure comment le temps d'exécution diminue avec l'augmentation du nombre de processus pour une taille de problème fixe.

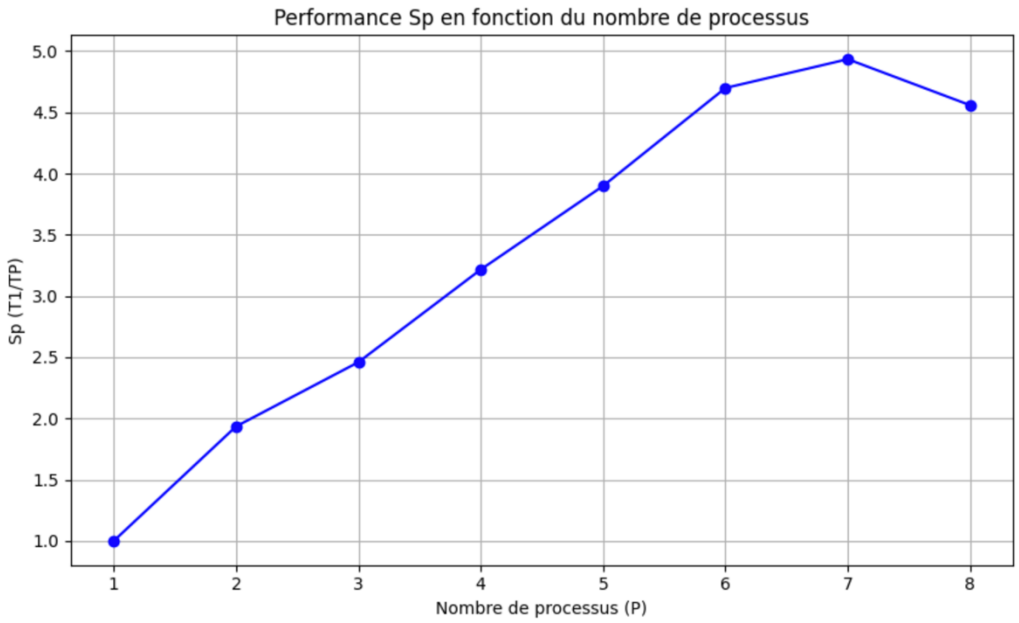
Voici un exemple :

Nombre de processus	Temps (ms)	Speedup	Efficacité
1		1.00	100%
2		1.92	96%
4		3.57	89%
8	160	6.25	78%

Lucas Vieira
INFO3-FA

Nous allons donc dans un premier temps, menez une expérience en prenant en nombre total d'itérations 16 000 000, j'ai décidé de mener 10 fois chaque expérience et pour le choix des workers, j'ai décidé de faire avec 1, 2, 3, 4, 5, 6, 7 et 8.

Cela me donne donc la courbe suivante :



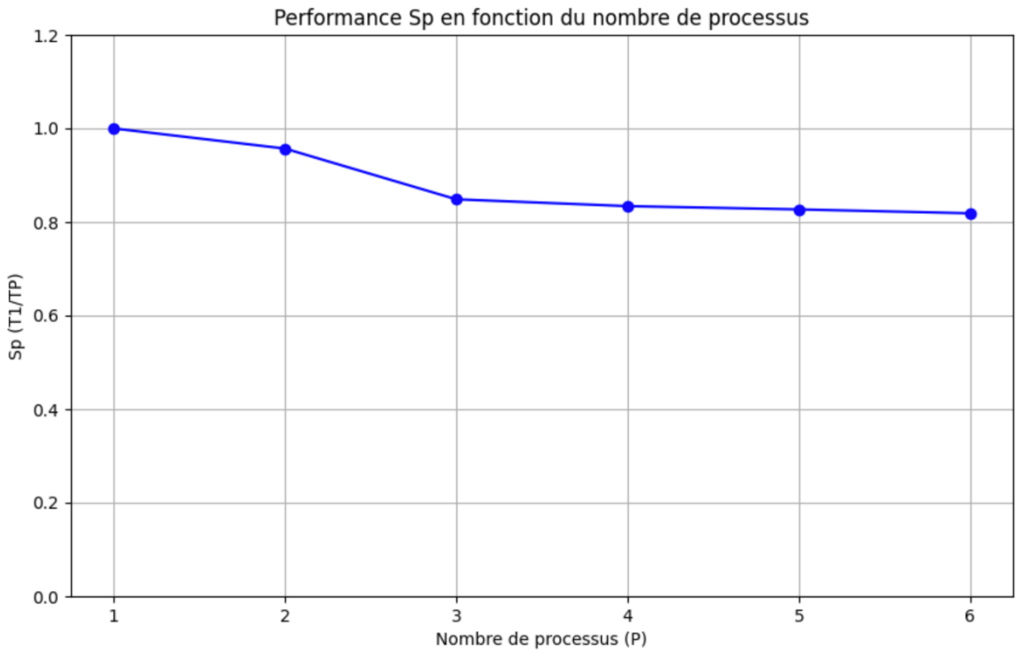
On peut donc constater que notre valeur maximale est de 5, ce qui signifie que la meilleure optimisation de notre machine est d'utiliser 7 workers car il s'agit de la meilleure valeur que l'on peut obtenir. Cela dit on remarque que de 1 à 4 on obtient un sp très proche du nombre de processus car nous utilisons les cœurs physiques et non les cœurs logiques.

III – Scalabilité faible :

La scalabilité faible consiste elle à mesurer comment le temps d'exécution varie quand on augmente à la fois le nombre de processus et la taille du problème proportionnellement.

Nombre de threads	Taille du problème	Temps (ms)
1	1M	1000
2	2M	1010
4	4M	1025
8	8M	1050

Nous allons donc dans un premier temps, menez une expérience en prenant en nombre total d'itérations 16 000 000, j'ai décidé de mener 10 fois chaque expérience et pour le choix des workers, j'ai décidé de faire avec 1, 2, 3, 4, 5, 6 car malheureusement la machine ne voulait pas aller jusqu'à 8. J'étais bloquer à partir de 6.



On constate que lorsqu'on augmente la taille du problème de manière proportionnelle, il vaut mieux réduire le nombre de processus afin d'augmenter le sp. Cela nous indique donc qu'il vaut mieux trouver un compromis entre le nombre de processus et la taille du problème pour chaque processus.

IV – Conclusion :

Pour conclure, nous pouvons dire que lorsque le problème augmente en taille il vaut mieux réduire le nombre de processus cependant lorsque la taille du problème reste constante il vaut mieux ajouter des workers dans la limite du nombre de cœurs physique.

Divers :

Cette section contient quelques éléments divers notamment le lien vers le répertoire Github ainsi que l'explication de l'utilisation de l'IA au sein de ce devoir.

J'ai utilisé l'IA afin de mieux comprendre le code au début puis je l'ai utilisé lorsque mon code ne fonctionnait pas donc je lui ai demandé de m'aider. Le modèle utiliser est Junie (Jetbrains).

Lucas Vieira
INFO3-FA

Mon compte rendu pour Pi.java est le readme du git. Lien vers le répertoire Github :
https://github.com/lucasv02/programmation-parrallele